



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-14 «Цифровые технологии обработки данных»

ОТЧЕТ ПО УЧЕБНОЙ ПРАКТИКЕ №13

на тему: «Тестирование и улучшение игры»

Выполнил:

Студент группы БСБО-43-24

Стасенок Юлия Анатольевна

Проверил:

к. т. н., доцент

Кашкин Евгений Владимирович

Москва, 2025 г.

Оглавление

Цель работы	3
Задание	3
Ход работы.....	3
Выводы	6
Список использованных источников	8

Цель работы

В ходе практического задания необходимо провести тестирование игры и зафиксировать все неисправности в её работе. Также необходимо устранить выявленные неисправности.

Задание

В ходе практического задания необходимо проверить:

1. все грейбокс-модели заменены на ассеты;
2. игрок не может проходить через другие объекты;
3. объекты, с которыми взаимодействует игрок, не проходят через другие объекты;
4. корректно происходит захват объектов;
5. при взаимодействии с врагом (другим активным объектом) происходит какое-то событие;
6. анимация настроена и работает корректно;
7. и др.

Ход работы

В ходе работы было проведено тестирование игры с целью выявления неисправностей в ней и дальнейшего их исправления. В таблице 11.1 представлен список протестированных элементов игры. Также напротив каждого из этих элементов игры указано, были ли выявлены неисправности в его работе.

Таблица 13.1. Список протестированных элементов игры

№	Что было протестировано	Были выявлены неисправности (Да/Нет)
1	Игровая карта	Нет
2	Враг	Да
3	Объекты, с которыми может взаимодействовать игрок	Да
4	UI	Да
5	События на объектах	Нет

В таблице 13.1 отражено, что у некоторых из элементов игры были выявлены одна или несколько неисправностей. В таблице 13.2 представлены

описания всех обнаруженных в игре неисправностей, а также указано, где они были зафиксированы. Также в таблице 13.2 указано то, как была выявлена каждая из неисправностей, и что было необходимо сделать с целью её исправления.

Таблица 13.2. Описание обнаруженных неисправностей

№	Описание неисправности	Где зафиксирована	Как была выявлена	Что необходимо сделать для исправления
1	Враг в состоянии смерти всё ещё вращался за игроком	Враг	Тестировщик обходил кругом вблизи врага в состоянии смерти	Отключение NavMeshAgent у врага, когда он входит в состояние смерти
2	Некорректная работа анимаций в случайный момент времени	Враг	Тестирование механик врага	Изменение типа анимации врага на гуманоидный (Humanoid)
3	Gaze Interactor взаимодействует с UI интерфейсом	UI	Тестирование Gaze Interactor	Отключение свойства UI Interaction в компоненте XR Gaze Interactor (см. листинг 13.1)
4	Точка прикрепления оружия к соответствующей руке настраивается только после взятия оружия. Оружие правильно прикрепляется только на второй попытке взятия	Объекты, с которыми может взаимодействовать игрок	Тестирование механик интерактивных объектов	Перегрузка метода OnSelectEntering у класса XRGrabInteractable (см. листинг 13.2), который будет настраивать точку прикрепления перед взятием оружия

В листинге кода 13.1 представлен фрагмент скрипта Akratit. В данной части скрипта врага происходит отключение компонента NawMeshAgent, отвечающего за поворот и перемещение, у врага при попадании его в состояние смерти. После перехода врага в состояние смерти его модель отключается, а активным становится префаб мёртвого врага, принимающий то же расположение и ротацию, что и враг в момент перехода в состояние смерти. Таким образом, враг при переходе в состояние смерти теряет возможность перемещаться и поворачиваться.

Указанный скрипт используется для исправления неисправности, которая состояла в том, что враг после перехода в состояние смерти продолжал поворачиваться в сторону игрока.

```
private void OnEnable()
{
    EventManager.Akratit.OnAkratitHit += GetDamage;
    EventManager.Save.OnSaveGame += SaveAkratitParameters;
}

private void OnDisable()
{
    EventManager.Akratit.OnAkratitHit -= GetDamage;
    EventManager.Save.OnSaveGame -= SaveAkratitParameters;
}

private void Start()
{
    LoadAkratitParameters();
}

public void SaveAkratitParameters() => SaveManager.SaveData(new
SaveData.AkratitData(this), savefileName);

public void LoadAkratitParameters()
{
    SaveData.AkratitData aktratitData =
    SaveManager.LoadData<SaveData.AkratitData>(savefileName);

    if (aktratitData == null)
    {
        return;
    }

    Vector3 position = new Vector3(aktratitData.position[0],
    aktratitData.position[1], aktratitData.position[2]);
    this.transform.position = position;

    Vector3 rotation = new Vector3(aktratitData.rotation[0],
    aktratitData.rotation[1], aktratitData.rotation[2]);
    this.transform.rotation = Quaternion.Euler(rotation);

    this.hp = aktratitData.hp;
    this.isDead = aktratitData.isDead;

    if (isDead)
    {
        GameObject dead = Instantiate(deadPrefab, this.transform.position,
    this.transform.rotation);
        this.gameObject.SetActive(false);
    }
}
```

Листинг 13.1. Фрагмент скрипта Akratit

В листинге 13.2 представлен скрипт Grab Stabilizer XR, в котором происходит перегрузка метода OnSelectEntering класса XRGrabInteractable, который присваивает точку прикрепления перед взятием предмета для исправления неисправности при захвате интерактивных объектов.

```
using UnityEngine;
using UnityEngine.XR.Interaction.Toolkit;
using UnityEngine.XR.Interaction.Toolkit.Interactables;

public class GrabStabilizerXR : XRGrabInteractable
{
    [SerializeField] Transform rightAttachTransform;
    [SerializeField] Transform leftAttachTransform;

    protected override void OnSelectEntering(SelectEnterEventArgs args)
    {
        if (args.interactorObject.transform.TryGetComponent(out LeftHand lefthand))
        {
            Debug.Log("Left hand");
            if (this.TryGetComponent(out XRGrabInteractable xRGrabInteractable))
            {
                xRGrabInteractable.attachTransform = leftAttachTransform;
            }
        }
        if (args.interactorObject.transform.GetComponent<RightHand>())
        {
            Debug.Log("Right Hand");
            if (this.TryGetComponent(out XRGrabInteractable xRGrabInteractable))
            {
                xRGrabInteractable.attachTransform = rightAttachTransform;
            }
        }
        base.OnSelectEntering(args);
    }

    args.interactableObject.transform.SetParent(args.interactorObject.transform);
}
}
```

Листинг 13.2. Скрипт Grab Stabilizer XR

Выводы

В ходе работы была проведена тестировка игры с целью проверки всех элементов игры для выявления неисправностей. Далее после выявления неисправностей в некоторых элементах игры были проведены необходимые для их устранения действия. Среди них изменение некоторых свойств объектов для устранения неисправностей анимаций и UI и изменения в скриптах некоторых объектов с целью реализации перегрузки методов и

отключения компонентов для устранения неисправностей в перемещении врага и взаимодействии с интерактивными объектами.

Список использованных источников

1. Working with humanoid animations [Электронный ресурс] URL: <https://docs.unity3d.com/530/Documentation/Manual/AvatarCreationandSetup.html> (дата обращения: 19.04.2025).